

◆ CS EDUCATION ◆

Q

SOLID

25.10.31
강철웅

교육 목표

1. SOLID 원칙의 개념과 이해
2. 좋은 코드와 나쁜 코드 구분
3. SOLID 사용시 주의사항

SOLID란?



- 로버트 마틴(Uncle Bob)이 제시한 객체지향 설계의 5가지 원칙으로, 유지보수하기 쉽고 확장 가능한 소프트웨어를 만들기 위한 지침이다.
- 각 원칙의 앞글자를 따서 SOLID라고 부르며, 실무에서 더 나은 코드를 작성하는 데 필수적인 개념들이다.
- SOLID 5개 원칙은 독립적으로 보이지만 서로 밀접하게 연결되어 있다. 하나의 원칙을 적용하면 자연스럽게 다른 원칙도 함께 적용되는 경우가 많다.

SOLID 5가지 원칙

SRP : 단일 책임 원칙

[Single Responsibility Principle]

OCP : 개방 폐쇄 원칙

[Open Closed Principle]

LSP : 리스코프 치환 원칙

[Liskov Subsitution Principle]

ISP : 인터페이스 분리 원칙

[Interface Segregaion Principle]

DIP : 의존성 역전 원칙

[Dependency Inversion Principle]

SOLID 이점

- ✅ 1. 코드의 유지보수성 향상 → 변경이 쉬워짐
- ✅ 2. 확장 가능한 구조 → 새로운 기능 추가가 용이
- ✅ 3. 안정성 보장 → 기존 코드를 건드리지 않고 확장
- ✅ 4. 버그 발생 가능성 감소
- ✅ 5. 테스트 용이성 증가

단일 책임 원칙 (SRP)

- "클래스는 하나의 책임만 가져야 하며, 단 하나의 변경 이유만을 가져야 한다"
- SRP는 SOLID 원칙 중에서도 가장 기본이 되는 원칙이다. 한 클래스가 너무 많은 일을 담당하면 코드가 복잡해지고 유지보수가 어려워진다.
- 예: User 클래스가 사용자 정보를 관리하고, UserReport 클래스가 사용자 보고서를 생성하도록 분리

❌ SRP의 나쁜 예시

```
// SRP 위반: 너무 많은 책임을 가짐
public class Computer {
    public void startComputer() {
        System.out.println("부팅 중...");
        checkHardware();
        loadOS();
        connectNetwork();
        System.out.println("컴퓨터 준비 완료");
    }

    private void checkHardware() {
        System.out.println("하드웨어 점검");
    }

    private void loadOS() {
        System.out.println("운영체제 로딩");
    }

    private void connectNetwork() {
        System.out.println("네트워크 연결");
    }
}
```

문제점

- Computer 클래스가 여러 가지 책임을 담당하고 있다.
 1. 하드웨어 점검 
 2. OS 로딩 
 3. 네트워크 연결 
- 예를 들어 하드웨어 점검 방식이 바뀌거나, OS 로딩 방식이 바뀌거나, 네트워크 연결 방식이 바뀔 때마다 Computer 클래스를 수정해야 한다.

✓ SRP의 좋은 예시

```
// 각자 하나의 책임만 가짐
class Hardware {
    void check() {
        System.out.println("하드웨어 점검");
    }
}

class OS {
    void load() {
        System.out.println("운영체제 로딩");
    }
}

class Network {
    void connect() {
        System.out.println("네트워크 연결");
    }
}

public class Computer {
    private final Hardware hardware = new Hardware();
    private final OS os = new OS();
    private final Network network = new Network();

    public void startComputer() {
        hardware.check();
        os.load();
        network.connect();
    }
}
```

개선점

- Hardware, OS, Network를 각각 별도 클래스로 분리
- 각 클래스는 하나의 책임만 가짐
- Computer 클래스는 세 클래스를 조합하여 순서대로 호출만 함
- 네트워크 연결 방식을 바꿔도 Hardware나 OS 클래스는 영향받지 않음
- 각 기능이 독립적으로 관리됨

개방-폐쇄 원칙 (OCP)

- "확장에는 열려 있고, 수정에는 닫혀 있어야 한다"
- OCP는 기존 코드를 수정하지 않고 새로운 기능을 추가할 수 있어야 한다는 원칙이다. 이는 소프트웨어의 확장성과 안정성을 동시에 보장하는 핵심적인 개념이다.
- 예: Shape 인터페이스를 만들어 Rectangle, Circle 등의 새로운 도형 클래스를 추가할 수 있도록 설계

❌ OCP의 나쁜 예시

```
// OCP 위반: 새로운 연산 추가 시 기존 코드 수정 필요
class Calculator {
    public double calculate(String operation, double a, double b) {
        if (operation.equals("add")) {
            return a + b;
        } else if (operation.equals("multiply")) {
            return a * b;
        }
        // 새로운 연산을 추가하려면 이 메서드를 수정해야 함
        return 0;
    }
}
```

문제점

- 나눗셈 추가 → calculate 메서드를 열어서 if-else 추가 필요
- 뺄셈 추가 → 또 메서드 수정 필요
- 기능 추가 시마다 기존 코드를 계속 수정해야 함
- if-else가 계속 늘어남
- 기존 코드를 건드리면 버그 위험 증가

✓ OCP의 좋은 예시

```
// 인터페이스로 확장 가능하게 설계
interface Operation {
    double execute(double a, double b);
}

class Addition implements Operation {
    public double execute(double a, double b) {
        return a + b;
    }
}

class Multiplication implements Operation {
    public double execute(double a, double b) {
        return a * b;
    }
}

// 새로운 연산 추가 시 기존 코드 수정 없이 확장 가능
class Division implements Operation {
    public double execute(double a, double b) {
        if (b != 0) {
            return a / b;
        }
        throw new IllegalArgumentException("0으로 나눌 수 없습니다");
    }
}

class Calculator {
    public double calculate(Operation operation, double a, double b) {
        return operation.execute(a, b);
    }
}
```

개선점

- Operation 인터페이스로 '연산'이라는 공통 규약 정의
- Addition, Multiplication, Division을 각각 별도 클래스로 구현
- Calculator는 Operation 인터페이스만 받음 (구체적인 연산은 신경 안 씀)
- 새로운 연산 추가 시 Calculator는 전혀 수정하지 않음
- 뺄셈 추가? → Subtraction 클래스만 생성, Calculator는 그대로
- 기존 기능의 안정성 보장

리스코프 치환 원칙 (LSP)

- "자식 클래스는 언제나 부모 클래스를 대체할 수 있어야 한다"
- LSP는 상속을 올바르게 사용하기 위한 원칙이다. 부모 클래스 타입의 객체를 자식 클래스 타입의 객체로 치환해도 프로그램이 정상적으로 동작해야 한다.
- 예: Bird 클래스의 fly() 메서드를 Penguin 클래스에서 무조건 구현하면 원칙을 위배 (펭귄은 날지 못함)

✖ LSP의 나쁜 예시

```
// LSP 위반 예시
class Bird {
    public void fly() {
        System.out.println("날아간다");
    }
}

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("타조는 날 수 없다");
    }
}

// 문제 상황
public class Zoo {
    public void makeBirdFly(Bird bird) {
        bird.fly(); // Bird 타입이면 당연히 날 수 있다고 가정
    }
}

Zoo zoo = new Zoo();
zoo.makeBirdFly(new Ostrich()); // ✨ 예외 발생! LSP 위반
```

문제점

- Bird 클래스의 fly() 메서드를 Ostrich가 오버라이드하여 예외를 던짐
- Zoo의 makeBirdFly는 "Bird면 날 수 있다"고 가정
- 타조를 넣으면 ✨ 예외 발생
- Bird 타입(부모클래스)을 Ostrich(자식클래스)로 대체할 수 없음
- 컴파일은 되지만 런타임에 터짐

✓ LSP의 좋은 예시

```
// 날 수 있는 능력을 인터페이스로 분리
interface Flyable {
    void fly();
}

// 날 수 있는 새들
class Sparrow implements Flyable {
    public void fly() {
        System.out.println("참새가 날아간다");
    }
}

class Eagle implements Flyable {
    public void fly() {
        System.out.println("독수리가 날아간다");
    }
}

class Pigeon implements Flyable {
    public void fly() {
        System.out.println("비둘기가 날아간다");
    }
}

// 날 수 없는 새들
class Ostrich {
    public void run() {
        System.out.println("타조가 달린다");
    }
}

class Penguin {
    public void swim() {
        System.out.println("펭귄이 헤엄친다");
    }
}
```

```
// 동물원
public class Zoo {
    public void makeFly(Flyable bird) {
        bird.fly(); // Flyable이면 100% 날 수 있음 보장
    }

    public static void main(String[] args) {
        Zoo zoo = new Zoo();

        // 날 수 있는 새들 - 모두 정상 작동 ✓
        zoo.makeFly(new Sparrow()); // "참새가 날아간다"
        zoo.makeFly(new Eagle());   // "독수리가 날아간다"
        zoo.makeFly(new Pigeon());   // "비둘기가 날아간다"

        // 날 수 없는 새들 - 타입이 안 맞아서 넣을 수 없음
        // zoo.makeFly(new Ostrich()); // ✗ 컴파일 에러!
        // zoo.makeFly(new Penguin()); // ✗ 컴파일 에러!

        // 날 수 없는 새들은 각자의 메서드 사용
        Ostrich ostrich = new Ostrich();
        ostrich.run(); // "타조가 달린다"

        Penguin penguin = new Penguin();
        penguin.swim(); // "펭귄이 헤엄친다"
    }
}
```

개선점

- Flyable 인터페이스로 '날 수 있는 것'을 분리
- Sparrow, Eagle, Pigeon은 Flyable 구현
- Ostrich는 아예 Flyable이 아님 (별도 클래스)
- 핵심: makeFly는 Flyable 타입을 받지만, 실제로는 자식(Sparrow 등)을 넣음
- 부모 타입 자리에 자식을 넣어도 모두 정상 작동
- 타조는 타입이 달라서 makeFly에 넣을 수 없음 (컴파일 에러로 미리 방지)

인터페이스 분리 원칙 (ISP)

- "특정 클라이언트에 맞는 인터페이스를 제공해야 하며, 불필요한 메서드가 포함된 거대한 인터페이스를 만들지 않아야 한다"
- ISP는 클라이언트가 자신이 사용하지 않는 메서드에 의존하게 되는 것을 방지하는 원칙이다. 거대한 인터페이스보다는 구체적이고 작은 인터페이스들이 낫다.
- 예: Printer 인터페이스가 `print()`, `scan()`, `fax()` 메서드를 모두 포함하면, 단순프린터 클래스는 `fax()` 기능이 불필요함 → `Printable`, `Scannable`, `Faxable` 인터페이스로 분리

✗ ISP의 나쁜 예시

```
// ISP 위반: 너무 많은 기능이 한 인터페이스에
interface Machine {
    void print();
    void scan();
    void fax();
}

class SimplePrinter implements Machine {
    public void print() {
        System.out.println("문서를 인쇄합니다");
    }

    public void scan() {
        // 사용하지 않는 기능이지만 구현해야 함
        throw new UnsupportedOperationException("스캔 기능 없음");
    }

    public void fax() {
        // 사용하지 않는 기능이지만 구현해야 함
        throw new UnsupportedOperationException("팩스 기능 없음");
    }
}
```

문제점

- Machine 인터페이스에 print, scan, fax가 모두 포함
- SimplePrinter는 인쇄만 필요한데 scan, fax도 구현해야 함
- 사용하지 않는 기능을 강제로 구현
- 불필요한 예외 처리 코드 필요

✓ ISP의 좋은 예시

```
// 기능별로 인터페이스 분리
interface Printable {
    void print();
}

interface Scannable {
    void scan();
}

interface Faxable {
    void fax();
}

class SimplePrinter implements Printable {
    public void print() {
        System.out.println("문서를 인쇄합니다");
    }
}

class MultiFunctionPrinter implements Printable, Scannable, Faxable {
    public void print() {
        System.out.println("문서를 인쇄합니다");
    }

    public void scan() {
        System.out.println("문서를 스캔합니다");
    }

    public void fax() {
        System.out.println("팩스를 전송합니다");
    }
}
```

개선점

- 기능별로 인터페이스 분리: Printable, Scannable, Faxable
- SimplePrinter는 Printable만 구현 (scan, fax 구현 불필요)
- MultiFunctionPrinter는 세 인터페이스 모두 구현
- 각 클래스가 실제로 사용하는 기능에만 의존
- 불필요한 예외 처리 제거

의존 역전 원칙 (DIP)

- "상위 모듈이 하위 모듈에 직접 의존하지 않고, 추상화를 통해 의존성을 줄여야 한다"
- DIP는 의존성의 방향을 역전시켜 더 유연한 설계를 만드는 원칙이다. 구체적인 구현체에 의존하는 것이 아니라 추상화(인터페이스)에 의존해야 한다.
- 예: MySQLDatabase 클래스에 직접 의존하는 대신, Database 인터페이스를 통해 MySQLDatabase, PostgreSQLDatabase 등을 사용할 수 있도록 설계

❌ DIP의 나쁜 예시

```
// DIP 위반: 구체적인 클래스에 직접 의존
class MySQLDatabase {
    public void connect() {
        System.out.println("MySQL에 연결");
    }
}

class DataService {
    private final MySQLDatabase database; // 구체적인 클래스에 직접 의존

    public DataService() {
        this.database = new MySQLDatabase(); // 강한 결합!
    }

    public void use() {
        database.connect();
    }
}
```

문제점

- DataService가 MySQLDatabase를 직접 의존
- 생성자에서 new MySQLDatabase()로 직접 생성
- PostgreSQL로 변경하려면 DataService 코드 수정 필요
- 강한 결합(tight coupling)

✓ DIP의 좋은 예시

```
// DIP 준수: 추상화에 의존
interface Database {
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("MySQL에 연결");
    }
}

class PostgreSQLDatabase implements Database {
    public void connect() {
        System.out.println("PostgreSQL에 연결");
    }
}

class DataService {
    private final Database database; // 인터페이스에 의존

    // 의존성 주입 (Dependency Injection)
    public DataService(Database database) {
        this.database = database;
    }

    public void use() {
        database.connect();
    }
}
```

개선점

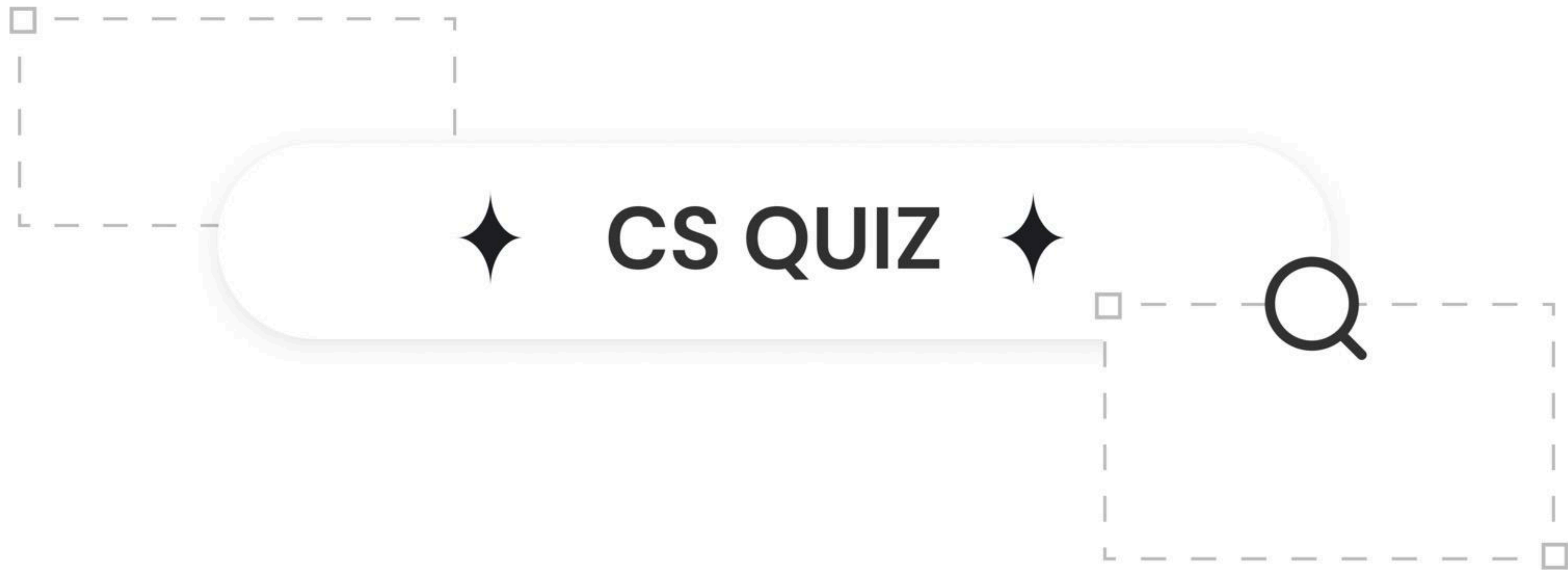
- Database 인터페이스 생성 (추상화)
- MySQLDatabase, PostgreSQLDatabase가 Database 구현
- DataService는 Database 인터페이스에만 의존
- 생성자로 Database를 주입받음 (의존성 주입)
- 핵심: DataService는 인터페이스에만 의존, 실제 사용 시 구현체(MySQL 등)를 주입
- DB 변경 시 DataService는 전혀 수정하지 않음
- 테스트용 가짜 객체(Mock) 생성 가능
- 느슨한 결합(loose coupling)으로 유연성 확보

⚠️ SOLID 원칙 적용 시 주의사항

- ✅ 1. 모든 상황에 무조건 적용할 필요는 없음
- ✅ 2. 프로젝트 규모와 복잡도를 고려
- ✅ 3. 간단한 코드를 불필요하게 복잡하게 만들지 말 것
- ✅ 4. "완벽한 설계"에 집착하지 말 것

언제 적용할까?

- ✓ 1. 코드가 자주 변경되는 부분
- ✓ 2. 여러 곳에서 재사용되는 부분
- ✓ 3. 테스트가 필요한 핵심 로직
- ✗ 4. 한 번만 사용되는 단순한 코드
- ✗ 5. 변경 가능성이 거의 없는 부분



Q1 다음 중 단어에 대한 설명이 알맞지 않은 것을 고르시오.

1. SRP(단일 책임 원칙) - 클래스는 하나의 책임, 하나의 변경 이유를 가진다.
2. OCP(개방 폐쇄 원칙) - 확장에는 열려 있고, 수정에는 닫혀 있다.
3. LSP(리스코프 치환 원칙) - 부모 클래스는 언제나 자식 클래스를 대체할 수 있어야 한다.
4. ISP(인터페이스 분리 원칙) - 특정 클라이언트에 맞는 인터페이스를 제공해야 하며, 불필요한 메서드가 포함된 거대한 인터페이스를 만들지 않아야 한다.
5. DIP(의존성 역전 원칙) - 상위 모듈이 하위 모듈에 직접 의존하지 않고, 추상화를 통해 의존성을 줄여야 한다.



Q1 다음 중 단어에 대한 설명이 알맞지 않은 것을 고르시오.

1. SRP(단일 책임 원칙) - 클래스는 하나의 책임, 하나의 변경 이유를 가진다.
 2. OCP(개방 폐쇄 원칙) - 확장에는 열려 있고, 수정에는 닫혀 있다.
 3. LSP(리스코프 치환 원칙) - 부모 클래스는 언제나 자식 클래스를 대체할 수 있어야 한다.
 4. ISP(인터페이스 분리 원칙) - 특정 클라이언트에 맞는 인터페이스를 제공해야 하며, 불필요한 메서드가 포함된 거대한 인터페이스를 만들지 않아야 한다.
 5. DIP(의존성 역전 원칙) - 상위 모듈이 하위 모듈에 직접 의존하지 않고, 추상화를 통해 의존성을 줄여야 한다.
- 3번. 자식 클래스가 언제나 부모 클래스를 대체할 수 있어야 함

Q2

다음은 DIP(의존 역전 원칙)에 대한 설명입니다. 빈칸을 알맞게 채워 주세요.

DIP는 의존성의 방향을 역전시켜서 더 유연한 설계를 만드는 원칙입니다. 따라서 개발자는 구체적인 구현체에 의존하지 않고, <빈칸>에 의존해야 합니다.



A2

다음은 DIP(의존 역전 원칙)에 대한 설명입니다. 빈칸을 알맞게 채워 주세요.

추상화, 인터페이스, Interface, Abstraction



Q3 다음은 어떤 코드에 대한 설명이다. 이 코드가 위반한 SOLID 원칙이 무엇인지 고르시오.

Cook이라는 클래스가 있다. 이 Cook 클래스는 음식 조리 방식인 boil, fry, steam과 같은 서로 다른 모든 조리 메서드를 하나의 클래스에서 모두 다룬다.

1. SRP
2. OCP
3. LSP
4. ISP
5. DIP



A3 다음은 어떤 코드에 대한 설명이다. 이 코드가 위반한 SOLID 원칙이 무엇인지 고르시오.

Cook이라는 클래스가 있다. 이 Cook 클래스는 음식 조리 방식인 boil, fry, steam과 같은 서로 다른 모든 조리 메서드를 하나의 클래스에서 모두 다룬다.

정답: SRP

SRP를 준수하려면, 조리 방식별로 BoilCooker, FryCooker, SteamCooker와 같이 클래스를 분리하여 각 클래스가 하나의 조리 방식이라는 단 하나의 책임만 가지도록 해야 합니다.



Q4 SOLID는 객체지향 설계의 몇 가지 원칙이며, 누가 제시했을까요?

1. 3가지 / 앨런 튜링
2. 4가지 / 마틴 파울러
3. 5가지 / 로버트 마틴
4. 6가지 / 켄트 벅



A4 SOLID는 객체지향 설계의 몇 가지 원칙이며, 누가 제시했을까요?

1. 3가지 / 앨런 튜링
2. 4가지 / 마틴 파울러
3. 5가지 / 로버트 마틴
4. 6가지 / 켄트 벅



Q5 SOLID 원칙을 과도하게 적용했을 때 생길 수 있는 문제는 무엇인가
요? (복수 선택)

1. 코드 복잡도 증가
2. 불필요한 추상화 남발
3. 유지보수성 향상
4. 오버엔지니어링 발생
5. 유연성 향상



A5 SOLID 원칙을 과도하게 적용했을 때 생길 수 있는 문제는 무엇인가
요? (복수 선택)

1. 코드 복잡도 증가
2. 불필요한 추상화 남발
3. 유지보수성 향상
4. 오버엔지니어링 발생
5. 유연성 향상



Q6

개방-폐쇄 원칙(OCP)에서 말하는 "확장에는 열려있고, 수정에는 닫혀있다"의 의미는?

1. 새로운 기능을 추가할 때 기존 코드를 수정해야 한다
2. 새로운 기능을 추가할 때 기존 코드를 수정하지 않고 확장할 수 있어야 한다
3. 코드를 수정할 수 없도록 잠가두어야 한다
4. 확장 기능은 별도 파일로 분리해야 한다



A6 개방-폐쇄 원칙(OCP)에서 말하는 "확장에는 열려있고, 수정에는 닫혀있다"의 의미는?

1. 새로운 기능을 추가할 때 기존 코드를 수정해야 한다
2. 새로운 기능을 추가할 때 기존 코드를 수정하지 않고 확장할 수 있어야 한다
3. 코드를 수정할 수 없도록 잠가두어야 한다
4. 확장 기능은 별도 파일로 분리해야 한다

1번 OCP의 정반대 개념이므로 틀리고,
3번 '수정이 닫혀있다'라는 말이 절대 수정 불가능이라는 뜻이 아님
4번 OCP는 구조적 설계 원칙이지, 물리적인 파일 배치 규칙이 아니다.

Q7

"자식 클래스는 언제나 부모 클래스를 대체할 수 있어야 한다"는 원칙의 이름은? (한글 또는 영문 약자)



A7

"자식 클래스는 언제나 부모 클래스를 대체할 수 있어야 한다"는 원칙의 이름은? (한글 또는 영문 약자)

리스코프 치환 원칙 (또는 LSP)



Q8

O / X 문제 : ISP(인터페이스 분리 원칙)를 준수할 때, 특정 클라이언트에 맞는 인터페이스를 제공하기 위해서 당장은 사용하지 않을 메서드까지 포함된 인터페이스로 만드는 것이 권장된다.



A8

O / X 문제 : ISP(인터페이스 분리 원칙)를 준수할 때, 특정 클라이언트에 맞는 인터페이스를 제공하기 위해서 당장은 사용하지 않을 메서드까지 포함된 인터페이스로 만드는 것이 권장된다.

정답: X (틀림).

ISP는 "불필요한 메서드가 포함된 거대한 인터페이스를 만들지 않아야한다"는 원칙으로, 클라이언트가 사용하지 않는 메서드에 의존하게 되는 것을 방지하는 원칙이다.



Q9

SOLID 원칙을 적용하여 소프트웨어를 설계할 때, “새로운 기능이 추가되더라도 기존 코드를 건드리지 않게 하여 얻을 수 있는 가장 중요한 이점”은 무엇입니까?
(SOLID 원칙의 이점에 언급된 5가지 핵심 단어 중 하나를 작성하세요.)



A9

SOLID 원칙을 적용하여 소프트웨어를 설계할 때, “새로운 기능이 추가되더라도 기존 코드를 건드리지 않게 하여 얻을 수 있는 가장 중요한 이점”은 무엇입니까?
(SOLID 원칙의 이점에 언급된 5가지 핵심 단어 중 하나를 작성하세요.)

정답: 안정성, Stability, 안정성 보장, 안정성보장

SOLID 원칙의 이점 중 하나는 "안정성 보장 - 기존 코드를 건드리지 않고 확장"할 수 있다는 점입니다. 이는 기능 추가 시 버그 위험을 줄이고 프로그램의 안정적인 동작을 보장합니다.



Q10



다음 중 의존 DIP(의존 역전 원칙)을 올바르게 적용한 경우는?
(pdf 자료의 20페이지 코드 참고)

1. DataService가 내부에서 new MySQLDatabase()로 객체를 직접 생성한다
2. DataServie가 Database 인터페이스를 의존하고, 생성자로 구현체를 주입받는다.
3. DataService가 여러 DB 연결 클래스를 if-else로 직접 관리한다
4. 테스트 코드에서 실제 MySQL 서버를 직접 연결해야 한다



A10



다음 중 의존 DIP(의존 역전 원칙)을 올바르게 적용한 경우는?
(pdf 자료의 20페이지 코드 참고)

1. DataService가 내부에서 new MySQLDatabase()로 객체를 직접 생성한다
2. DataServie가 Database 인터페이스를 의존하고, 생성자로 구현체를 주입받는다.
3. DataService가 여러 DB 연결 클래스를 if-else로 직접 관리한다
4. 테스트 코드에서 실제 MySQL 서버를 직접 연결해야 한다

정답: 2번

1. 구체 클래스(MySQLDatabase)를 직접 생성하여 강하게 결합됨 → DIP 위반
2. 인터페이스(Database)에 의존하고 생성자로 주입받아 느슨한 결합 → DIP 준수
3. if-else로 여러 DB를 직접 관리하면 구체 클래스들에 의존 → DIP 위반
4. 실제 MySQL 서버를 직접 연결하면 테스트 불가능 → DIP의 장점(Mock 사용) 활용 못함



◆ CS EDUCATION ◆

🔍

감사합니다 :D

12기 CS 교육 장관부

